

SQL and SparkSQL

Overview

- Reminders
- The Big Picture
- SQL Basics
- SparkSQL - Context
- JDBC Connectors
- Spark Runtime
- Basic Data Management Operations in SparkSQL
- Take home exercise (Ungraded)
- Readings

Reminders

- Course entry-survey questions are posted on Canvas
- Homework-1 is due next Thursday 9/10.
- This week's recitation is happening on Friday 2pm ET.

Do I have the right python and pyspark configurations?

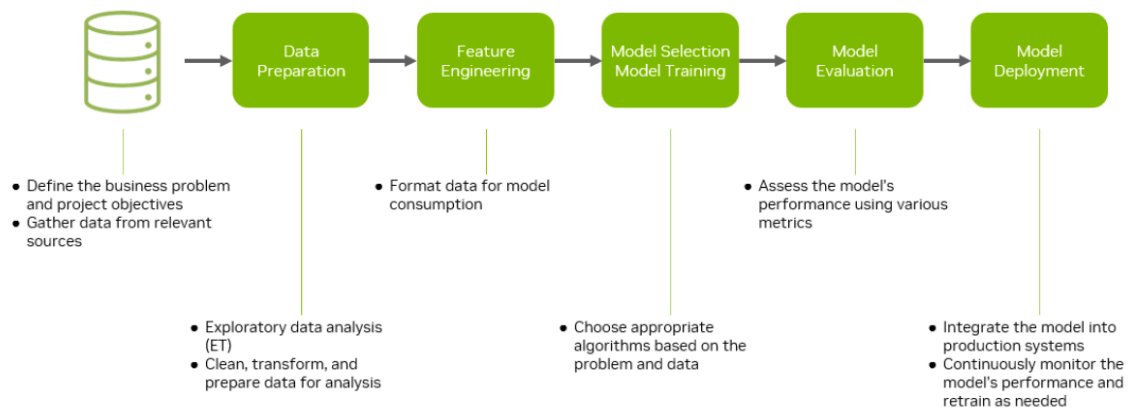
Several engineers report issues with running Spark code and in many cases, these issues are due to incorrect environment configurations. To avoid these issues, check your environment variables at the beginning of each session.

```
In [ ]: #Check your environment variables
import sys
import pyspark
import os

print("Python:", sys.executable)
print("PySpark version:", pyspark.__version__)
print("PySpark path:", pyspark.__file__)
print("SPARK_HOME:", os.environ.get("SPARK_HOME"))
```

Do you remember the Machine Learning Modeling Process that was introduced in our first lecture?

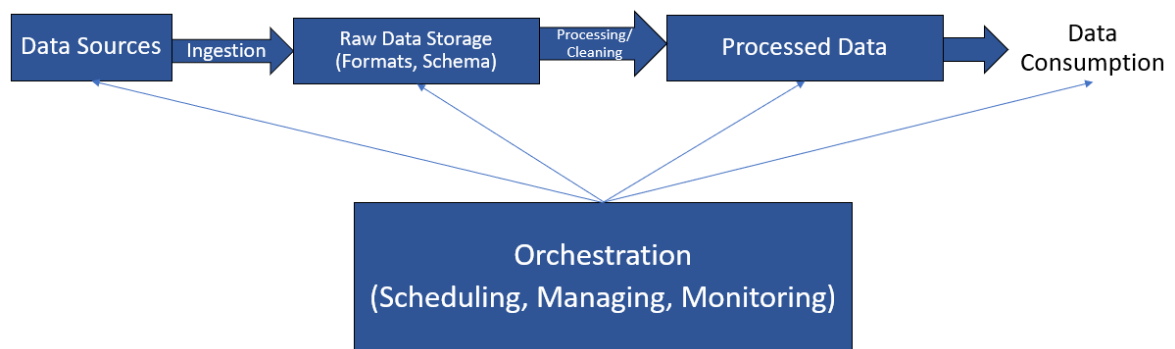
Machine learning has much more to it than just the training.



The Machine Learning Modeling Process

The Big Picture - Data Engineering Pipeline

The big picture in machine learning modeling lies in the **Data Engineering pipeline concept**. A data pipeline consists of a series of connected processes that move the data from one point to another, possibly transforming the data along the way. **The data engineering pipeline covers up to the Data preparation phase of the machine learning modeling process.**

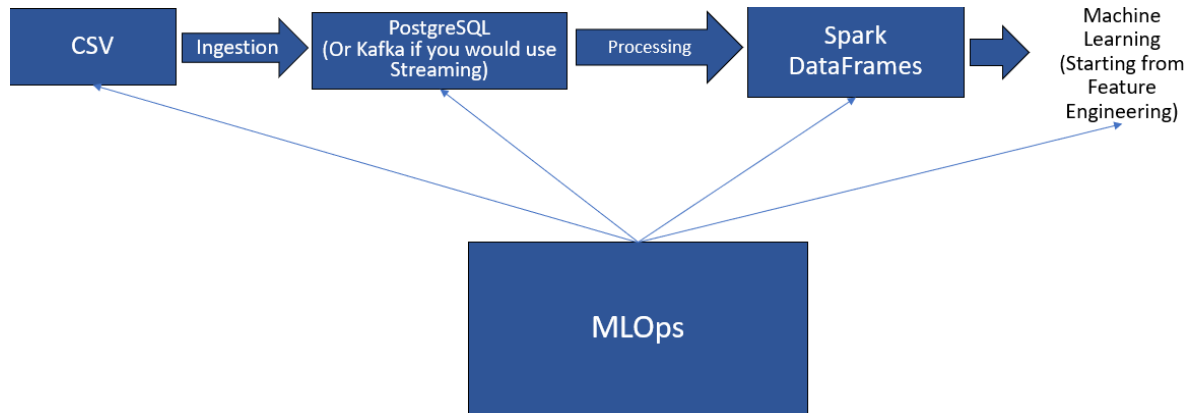


The General Processes in Data Engineering Pipeline

Ingested data can be stored in a better format by being stored into:

- Data warehouses.
- Data lakes.
- NoSQL databases.
- Relational databases.

In our course, we will use the following pipeline:



We will discuss each step in detail throughout the course

Relational Databases - One Option to Store Your Ingested Data

- A relational database (or RDB) is a structure for organizing information that one may think of initially as a supercharged version of a data frame.
- RDBs are commonly manipulated and queried using **SQL: Structured Query Language**.
- SQL is **case insensitive**.
- In an RDB, the basic unit of data storage is the **table (or relation)**. Rows are rows, but sometimes dubbed tuples or records, and columns are sometimes dubbed attributes. (Fields are specific cells within a given row.)

Postgres

- There are many implementations of SQL (SEE-kwule): Oracle, MySQL, SQLite, etc.
- In this class, we will make use of PostgreSQL, or more simply Postgres. It is open source, with Mac, Windows, and Linux versions available. You should download and install it!
- Postgres will set up a server on your machine. When you install Postgres, a superuser account "postgres" will be set up, with a password specified by you. Make this a good one, i.e., not just "postgres." (That's because hackers like to look for lazily set up postgres accounts.)

Categories of SQL Commands

Types of SQL Commands			
DDL	DML	DCL	TCL
CREATE ALTER DROP TRUNCATE RENAME	SELECT INSERT UPDATE DELETE MERGE	GRANT REVOKE	COMMIT ROLLBACK SAVEPOINT

SQL Command Categories

Data Definition Language (DDL)

DDL takes care of table structures. We will look into the most popular DDLs for creating, updating or dropping a table.

1. Creating a Table

Assuming you will need to create and populate a table to host your data.

The command is

```
CREATE TABLE IF NOT EXISTS < < name > >  
(<< column 1 >>> << type 1 >> << constraint 1 >>, ... , << multi-column constraint(s) >>);
```

Here is a simple example:

```
CREATE TABLE IF NOT EXISTS products(  
    product_id SERIAL,  
    label TEXT,  
    price decimal,  
    inventory INTEGER  
);
```

This creates a four-column table that contains a label for each product, its price, and the current number of each product available.

① (Be careful! You may not have the permissions to create tables - depending on your Postgres Installation-). To fix it, grant your user the proper permissions using this command **GRANT CREATE ON SCHEMA public TO postgres;**

Available Data Types

2. Alter a Table

If you wish to add or delete an entire column, or rename an column, or change constraints, etc., you would “update” your table using alter table.

The command is

```
ALTER TABLE < < name > >  
<< action >> ;
```

Here are two simple examples:

```
ALTER TABLE products ADD COLUMN rating REAL DEFAULT 0.0;
```

```
ALTER TABLE products DROP COLUMN rating;
```

3. Drop a Table

To remove a table in its entirety:

```
DROP TABLE <<name>>;
```

To check that the table is removed, look for it in the Table list on PgAdmin4.

Data Manipulation Language (DML)

DML takes care of data in the tables. We will look into the most popular DMLs for data management.

1. Insert Values

There are a few ways to insert data into a SQL table. We'll show how to do it row-by-row here, and then utilize select to create bigger tables next week.

To populate the table one row at a time:

```
INSERT INTO << table >> (<< column i >>, << column j >>,...) VALUES
(<< value i >>, << value j >>,...),
...
(<< value i >>, << value j >>,...);
```

If you leave out a column, then the data there will be missing (and can be added later with **UPDATE TABLE**) or will have a default value. Note that any column with data type SERIAL has default behavior: it will auto-increment).

```
INSERT INTO products (label,price,inventory) VALUES
('kirk action figure',50,13),
('spock action figure',40,22);
</div> The product_id, being of type SERIAL, will take on the values 1, then 2.
```

2. Select: Querying a Database

The **SELECT** command is how we query a database. It is a versatile and powerful command!

A shortened definition that highlights elements of the syntax that are important in the context of this class is:

```
SELECT
  <<column1, column2, columnn>>
FROM <<table>>
[WHERE <<condition>>]
[GROUP BY <<expression>>]
[HAVING <<condition>>]
[ORDER BY <<expression>>];
```

Example:

```
SELECT product_id, label, price
FROM products
WHERE price >= 10;
```

3. Update Table Values

The **UPDATE** command allows us to modify values in table cells.

```
UPDATE << table >>
  SET <<column1>> = << new value 1 >> ,
  SET <<column2>> = << new value 2 >>,
  ...
  WHERE <<row condition >>;
</div>
```

Think of the **WHERE < row condition >** as being like a call to the **which()** function in R: in it, you set a range of values for one of the table columns, and thereby select which rows to update.

```
UPDATE products
  set price = 100
  where price >= 45;
```

(Note that when you look at an updated table, the serial data may not be displayed in numeric order, i.e., the rows may be rearranged. This is OK.)

4. Delete Rows

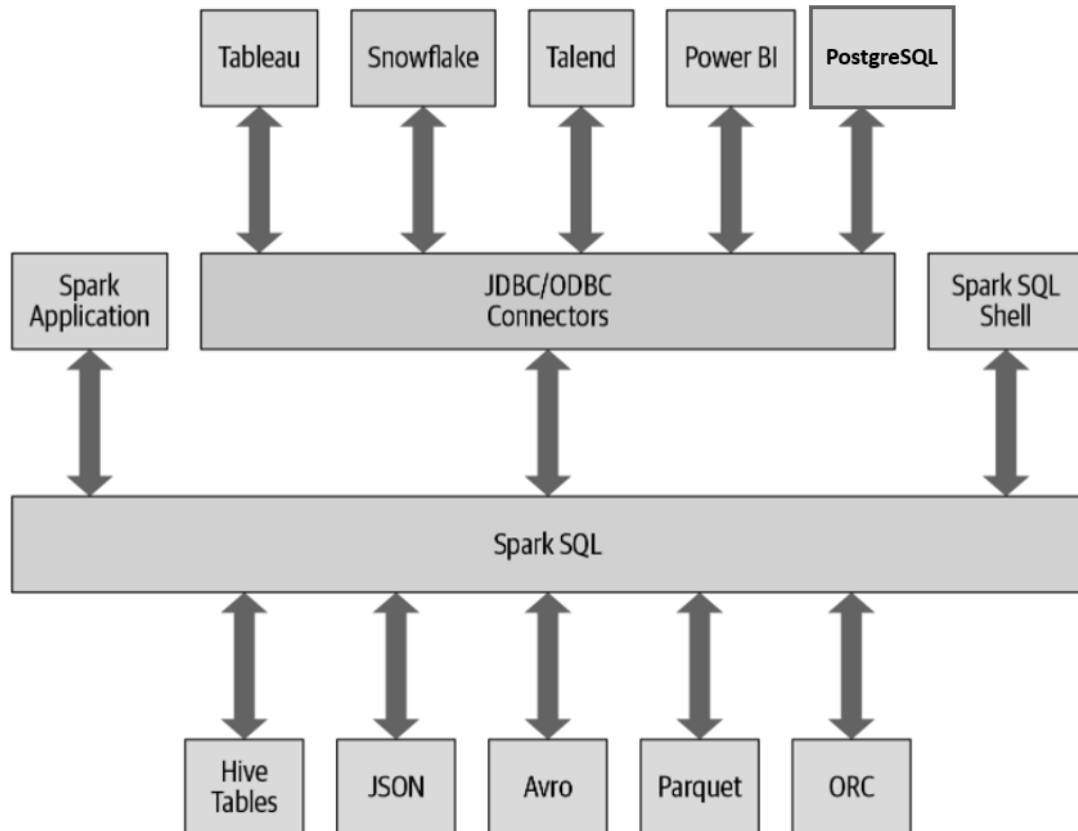
To remove one or more entries from a table:

```
DELETE FROM << table >>
  WHERE << condition >>;
```

SparkSQL - Context

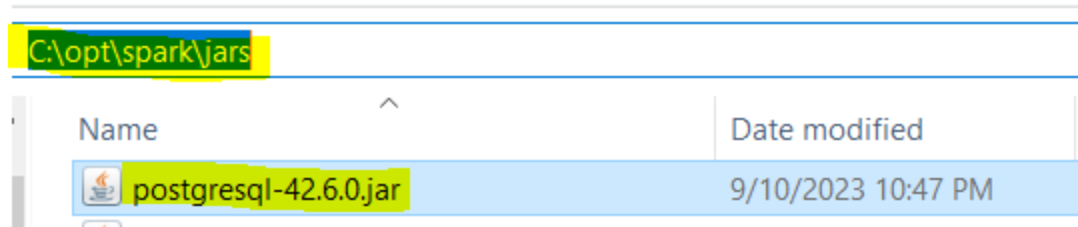
- SQL provides the main knowledge to ingest data into PostgreSQL tables.
- SQL doesn't support big-data processing on its own. So, in order to conduct SQL on big data, one good way to do it is to use Spark SQL

However, Spark uses "connectors" to integrate its operation with external platforms/applications (e.g. PostgreSQL).



Install JDBC Connector

- Watch the tutorial videos posted on Canvas for detailed configuration
- Install the JDBC driver that matches your Java Version from <https://jdbc.postgresql.org/download/>
- Place the installed **JAR file** into your **SPARK_HOME** directory under **jars** folder.
- Close all the terminals and restart Jupyter notebook over in a new terminal (or Restart Your Machine)



The Non-SparkSQL way to Initialize the Application

Continue to use this approach if you don't need to interface with Database

```
In [ ]: # Uncomment the following lines if you are using Windows!

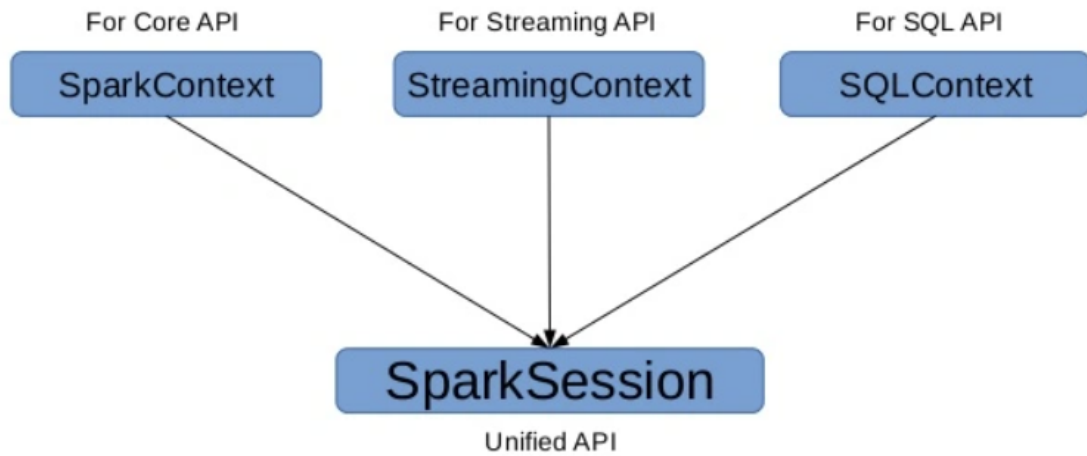
#import findspark
#findspark.init()
#findspark.find()

import pyspark
from pyspark.sql import SparkSession

spark = SparkSession.builder.master("local[*]").appName('SparkTest').getOrCreate()
```

Spark Runtime

- SparkContext is used to help you access hardware-level and some software-level configurations for your Spark application.
- Most Spark Components have their own Contexts (e.g. StreamingContext for streaming purposes).
- Starting Spark 2.x, SparkSession was created to run applications in an easier way than SparkContext.
- You may want to access the context object in order to specify the hardware-level or software-level configurations you need to specify



Contexts and SparkSession

Initialize SparkSQL Application - Create SQL Context

```
In [ ]: # Uncomment the following lines if you are using Windows!

#import findspark
#findspark.init()
#findspark.find()

import pyspark

from pyspark.sql import SparkSession
from pyspark import SparkContext, SQLContext

appName = "Big Data Analytics"
master = "local"

# Create Configuration object for Spark.
conf = pyspark.SparkConf()\
    .set('spark.driver.host','127.0.0.1')\
    .setAppName(appName)\
    .setMaster(master)

# Create Spark Context with the new configurations rather than relying on the default
sc = SparkContext.getOrCreate(conf=conf)

# You need to create SQL Context to conduct some database operations like what we will
sqlContext = SQLContext(sc)

# If you have SQL context, you create the session from the Spark Context
spark = sqlContext.sparkSession.builder.getOrCreate()
```

When to use plain SQL vs SparkSQL?

1. Use plain SQL to conduct DDL and DCL related operations and store them in *.sql files. These operations are not impacted by the data size
2. Use SparkSQL to conduct DML related operations.

Download the Dataset

```
In [5]: !python -m wget https://www.andrew.cmu.edu/user/mfarag/763/KDDTrain+.txt
```

Saved under KDDTrain+ (2).txt

Read-in the Dataset

```
In [6]: # Load data from csv to a dataframe on a local machine.
# header=False means the first row is not a header
# sep=',' means the column are seperated using ','
col_names = ["duration", "protocol_type", "service", "flag", "src_bytes",
             "dst_bytes", "land", "wrong_fragment", "urgent", "hot", "num_failed_logins",
             "logged_in", "num_compromised", "root_shell", "su_attempted", "num_root",
             "num_file_creations", "num_shells", "num_access_files", "num_outbound_cmds",
             "is_host_login", "is_guest_login", "count", "srv_count", "error_rate",
             "srv_error_rate", "rerror_rate", "srv_rerror_rate", "same_srv_rate",
             "diff_srv_rate", "srv_diff_host_rate", "dst_host_count", "dst_host_srv_count",
             "dst_host_same_srv_rate", "dst_host_diff_srv_rate", "dst_host_same_src_port_rate",
             "dst_host_srv_diff_host_rate", "dst_host_error_rate", "dst_host_srv_error_rate",
             "dst_host_rerror_rate", "dst_host_srv_rerror_rate", "classes", "difficulty_level"]

df = spark.read.csv("KDDTrain+.txt", header=False, inferSchema=True).toDF(*col_names)
```

Print Schema

```
In [7]: df.printSchema()
```

```
root
|-- duration: integer (nullable = true)
|-- protocol_type: string (nullable = true)
|-- service: string (nullable = true)
|-- flag: string (nullable = true)
|-- src_bytes: integer (nullable = true)
|-- dst_bytes: integer (nullable = true)
|-- land: integer (nullable = true)
|-- wrong_fragment: integer (nullable = true)
|-- urgent: integer (nullable = true)
|-- hot: integer (nullable = true)
|-- num_failed_logins: integer (nullable = true)
|-- logged_in: integer (nullable = true)
|-- num_compromised: integer (nullable = true)
|-- root_shell: integer (nullable = true)
|-- su_attempted: integer (nullable = true)
|-- num_root: integer (nullable = true)
|-- num_file_creations: integer (nullable = true)
|-- num_shells: integer (nullable = true)
|-- num_access_files: integer (nullable = true)
|-- num_outbound_cmds: integer (nullable = true)
|-- is_host_login: integer (nullable = true)
|-- is_guest_login: integer (nullable = true)
|-- count: integer (nullable = true)
|-- srv_count: integer (nullable = true)
|-- error_rate: double (nullable = true)
|-- srv_error_rate: double (nullable = true)
|-- rerror_rate: double (nullable = true)
|-- srv_rerror_rate: double (nullable = true)
|-- same_srv_rate: double (nullable = true)
|-- diff_srv_rate: double (nullable = true)
|-- srv_diff_host_rate: double (nullable = true)
```

```
|-- dst_host_count: integer (nullable = true)
|-- dst_host_srv_count: integer (nullable = true)
|-- dst_host_same_srv_rate: double (nullable = true)
|-- dst_host_diff_srv_rate: double (nullable = true)
|-- dst_host_same_src_port_rate: double (nullable = true)
|-- dst_host_srv_diff_host_rate: double (nullable = true)
|-- dst_host_serror_rate: double (nullable = true)
|-- dst_host_srv_serror_rate: double (nullable = true)
|-- dst_host_rerror_rate: double (nullable = true)
|-- dst_host_srv_rerror_rate: double (nullable = true)
|-- classes: string (nullable = true)
|-- difficulty_level: integer (nullable = true)
```

Write Data to Tables in SparkSQL

What is the SQL command represented in this code snippet?

```
In [8]: # update the options with your db user and password

db_properties={}
db_properties['username']="postgres"
db_properties['password']="bigdata"
db_properties['url']="jdbc:postgresql://localhost:5432/postgres"
db_properties['table']="intrusion"
db_properties['driver']="org.postgresql.Driver"

df.write.format("jdbc")\
.mode("overwrite")\
.option("url", db_properties['url'])\
.option("dbtable", db_properties['table'])\
.option("user", db_properties['username'])\
.option("password", db_properties['password'])\
.option("driver", db_properties['driver'])\
.save()
```

Spark Data Write Modes

In Spark, there are several modes for writing your dataframe data to external data source. In the previous example, we used "overwrite" but it's not the only option!

- **append**: Append contents of this dataframe to existing table's data. **Append** simulates the process of **inserting new records**.
- **overwrite**: Overwrite existing data. It recreates the table with all the data. Use it with cautious.
- **ignore**: Silently ignore this operation if data already exists.
- **error** (default case): Throw an exception if data already exists.

If you face any issues with the overwrite mode, try the following solutions:

1. Try **ALTER SCHEMA public OWNER TO postgres;** in your PgAdmin (or SQL file)
OR
2. Create the table via SQL and use the **append** mode (to be discussed).

Now, read the data back!

```
In [9]: df_read = sqlContext.read.format("jdbc")\
.option("url", db_properties['url'])\
.option("dbtable", db_properties['table'])\
.option("user", db_properties['username'])\
.option("password", db_properties['password'])\
.option("Driver", db_properties['driver'])\
.load()

df_read.printSchema()

root
 |-- duration: integer (nullable = true)
 |-- protocol_type: string (nullable = true)
 |-- service: string (nullable = true)
```

```

|-- flag: string (nullable = true)
|-- src_bytes: integer (nullable = true)
|-- dst_bytes: integer (nullable = true)
|-- land: integer (nullable = true)
|-- wrong_fragment: integer (nullable = true)
|-- urgent: integer (nullable = true)
|-- hot: integer (nullable = true)
|-- num_failed_logins: integer (nullable = true)
|-- logged_in: integer (nullable = true)
|-- num_compromised: integer (nullable = true)
|-- root_shell: integer (nullable = true)
|-- su_attempted: integer (nullable = true)
|-- num_root: integer (nullable = true)
|-- num_file_creations: integer (nullable = true)
|-- num_shells: integer (nullable = true)
|-- num_access_files: integer (nullable = true)
|-- num_outbound_cmds: integer (nullable = true)
|-- is_host_login: integer (nullable = true)
|-- is_guest_login: integer (nullable = true)
|-- count: integer (nullable = true)
|-- srv_count: integer (nullable = true)
|-- serror_rate: double (nullable = true)
|-- srv_serror_rate: double (nullable = true)
|-- rerror_rate: double (nullable = true)
|-- srv_rerror_rate: double (nullable = true)
|-- same_srv_rate: double (nullable = true)
|-- diff_srv_rate: double (nullable = true)
|-- srv_diff_host_rate: double (nullable = true)
|-- dst_host_count: integer (nullable = true)
|-- dst_host_srv_count: integer (nullable = true)
|-- dst_host_same_srv_rate: double (nullable = true)
|-- dst_host_diff_srv_rate: double (nullable = true)
|-- dst_host_same_src_port_rate: double (nullable = true)
|-- dst_host_srv_diff_host_rate: double (nullable = true)
|-- dst_host_serror_rate: double (nullable = true)
|-- dst_host_srv_serror_rate: double (nullable = true)
|-- dst_host_rerror_rate: double (nullable = true)
|-- dst_host_srv_rerror_rate: double (nullable = true)
|-- classes: string (nullable = true)
|-- difficulty_level: integer (nullable = true)

```

In [10]: `df_read.show(1, vertical=True)`

```

-RECORD 0-----
duration          | 0
protocol_type     | tcp
service           | http
flag              | REJ
src_bytes         | 0
dst_bytes         | 0
land              | 0
wrong_fragment    | 0
urgent            | 0
hot               | 0
num_failed_logins | 0
logged_in         | 0
num_compromised   | 0
root_shell        | 0
su_attempted      | 0
num_root          | 0
num_file_creations | 0
num_shells        | 0
num_access_files  | 0
num_outbound_cmds | 0
is_host_login     | 0

```

is_guest_login	0
count	1
srv_count	1
serror_rate	0.0
srv_serror_rate	0.0
rerror_rate	1.0
srv_rerror_rate	1.0
same_srv_rate	1.0
diff_srv_rate	0.0
srv_diff_host_rate	0.0
dst_host_count	34
dst_host_srv_count	251
dst_host_same_srv_rate	1.0
dst_host_diff_srv_rate	0.0
dst_host_same_src_port_rate	0.03
dst_host_srv_diff_host_rate	0.06
dst_host_serror_rate	0.0
dst_host_srv_serror_rate	0.0
dst_host_rerror_rate	1.0
dst_host_srv_rerror_rate	0.98
classes	normal
difficulty_level	21

only showing top 1 row

Selecting Columns

Keep in mind, you can chain functions here!

```
In [11]: # Selecting a single column
subset_df = df.select("classes")

# Selecting multiple columns and display them on the flyabs
df.select("protocol_type", "duration", "flag", "classes").show(1, vertical=True)

-RECORD 0-----
protocol_type | tcp
duration      | 0
flag          | SF
classes       | normal
only showing top 1 row
```

Filtering Data

Filtering allows you to subset a DataFrame based on specific conditions.

```
In [12]: # Filter rows where a condition is met
df_new = df_read.filter(df_read["classes"] == 'normal')
df_new.show(1, vertical=True)
```

```
-RECORD 0-----
duration      | 0
protocol_type | udp
service       | domain_u
flag          | SF
src_bytes     | 43
dst_bytes     | 135
land          | 0
wrong_fragment | 0
urgent        | 0
hot           | 0
num_failed_logins | 0
logged_in     | 0
num_compromised | 0
root_shell    | 0
su_attempted  | 0
num_root      | 0
num_file_creations | 0
num_shells    | 0
num_access_files | 0
num_outbound_cmds | 0
is_host_login | 0
is_guest_login | 0
count         | 279
srv_count     | 278
serror_rate   | 0.0
srv_serror_rate | 0.0
rerror_rate   | 0.0
srv_rerror_rate | 0.0
same_srv_rate | 1.0
diff_srv_rate | 0.01
srv_diff_host_rate | 0.0
dst_host_count | 244
dst_host_srv_count | 209
dst_host_same_srv_rate | 0.86
dst_host_diff_srv_rate | 0.01
```

```

dst_host_same_src_port_rate | 0.0
dst_host_srv_diff_host_rate | 0.0
dst_host_serror_rate       | 0.0
dst_host_srv_serror_rate   | 0.0
dst_host_rerror_rate       | 0.0
dst_host_srv_rerror_rate   | 0.0
classes                     | normal
difficulty_level           | 18
only showing top 1 row

```

```

In [13]: # Filter using SQL-like syntax
df_read.filter("count > 1").show(1,vertical=True)

```

```

-RECORD 0-----
duration                | 0
protocol_type           | tcp
service                 | private
flag                    | S0
src_bytes                | 0
dst_bytes                | 0
land                    | 0
wrong_fragment          | 0
urgent                  | 0
hot                     | 0
num_failed_logins       | 0
logged_in               | 0
num_compromised         | 0
root_shell              | 0
su_attempted            | 0
num_root                | 0
num_file_creations      | 0
num_shells              | 0
num_access_files        | 0
num_outbound_cmds       | 0
is_host_login           | 0
is_guest_login          | 0
count                   | 147
srv_count               | 18
serror_rate             | 1.0
srv_serror_rate         | 1.0
rerror_rate             | 0.0
srv_rerror_rate         | 0.0
same_srv_rate           | 0.12
diff_srv_rate           | 0.07
srv_diff_host_rate      | 0.0
dst_host_count          | 255
dst_host_srv_count      | 18
dst_host_same_srv_rate  | 0.07
dst_host_diff_srv_rate  | 0.07
dst_host_same_src_port_rate | 0.0
dst_host_srv_diff_host_rate | 0.0
dst_host_serror_rate    | 1.0
dst_host_srv_serror_rate | 1.0
dst_host_rerror_rate    | 0.0
dst_host_srv_rerror_rate | 0.0
classes                 | neptune
difficulty_level        | 21
only showing top 1 row

```

Renaming Columns

To rename columns in a DataFrame, you can use the `withColumnRenamed()` method.

```
In [14]: # Filter rows where a condition is met
df_read.withColumnRenamed("src_bytes", "source_bytes").printSchema()

root
|-- duration: integer (nullable = true)
|-- protocol_type: string (nullable = true)
|-- service: string (nullable = true)
|-- flag: string (nullable = true)
|-- source_bytes: integer (nullable = true)
|-- dst_bytes: integer (nullable = true)
|-- land: integer (nullable = true)
|-- wrong_fragment: integer (nullable = true)
|-- urgent: integer (nullable = true)
|-- hot: integer (nullable = true)
|-- num_failed_logins: integer (nullable = true)
|-- logged_in: integer (nullable = true)
|-- num_compromised: integer (nullable = true)
|-- root_shell: integer (nullable = true)
|-- su_attempted: integer (nullable = true)
|-- num_root: integer (nullable = true)
|-- num_file_creations: integer (nullable = true)
|-- num_shells: integer (nullable = true)
|-- num_access_files: integer (nullable = true)
|-- num_outbound_cmds: integer (nullable = true)
|-- is_host_login: integer (nullable = true)
|-- is_guest_login: integer (nullable = true)
|-- count: integer (nullable = true)
|-- srv_count: integer (nullable = true)
|-- error_rate: double (nullable = true)
|-- srv_error_rate: double (nullable = true)
|-- rerror_rate: double (nullable = true)
|-- srv_rerror_rate: double (nullable = true)
|-- same_srv_rate: double (nullable = true)
|-- diff_srv_rate: double (nullable = true)
|-- srv_diff_host_rate: double (nullable = true)
|-- dst_host_count: integer (nullable = true)
|-- dst_host_srv_count: integer (nullable = true)
|-- dst_host_same_srv_rate: double (nullable = true)
|-- dst_host_diff_srv_rate: double (nullable = true)
|-- dst_host_same_src_port_rate: double (nullable = true)
|-- dst_host_srv_diff_host_rate: double (nullable = true)
|-- dst_host_error_rate: double (nullable = true)
|-- dst_host_srv_error_rate: double (nullable = true)
|-- dst_host_rerror_rate: double (nullable = true)
|-- dst_host_srv_rerror_rate: double (nullable = true)
|-- classes: string (nullable = true)
|-- difficulty_level: integer (nullable = true)
```

Transforming Data

Data transformation involves applying functions or expressions to columns in a DataFrame to create new columns or modify existing ones.

```
In [15]: # Example 1
from pyspark.sql.functions import col

# Create a new column
df_with_double_difficulty_level = df_read.withColumn("double_difficulty_level", col("difficulty_level") * 2)

df_with_double_difficulty_level.show(1, vertical=True)
```

-RECORD 0-----	
duration	0
protocol_type	tcp
service	http
flag	SF
src_bytes	239
dst_bytes	2265
land	0
wrong_fragment	0
urgent	0
hot	0
num_failed_logins	0
logged_in	1
num_compromised	0
root_shell	0
su_attempted	0
num_root	0
num_file_creations	0
num_shells	0
num_access_files	0
num_outbound_cmds	0
is_host_login	0
is_guest_login	0
count	17
srv_count	17
serror_rate	0.0
srv_serror_rate	0.0
rerror_rate	0.0
srv_rerror_rate	0.0
same_srv_rate	1.0
diff_srv_rate	0.0
srv_diff_host_rate	0.0
dst_host_count	255
dst_host_srv_count	255
dst_host_same_srv_rate	1.0
dst_host_diff_srv_rate	0.0
dst_host_same_src_port_rate	0.0
dst_host_srv_diff_host_rate	0.0
dst_host_serror_rate	0.0
dst_host_srv_serror_rate	0.0
dst_host_rerror_rate	0.0
dst_host_srv_rerror_rate	0.0
classes	normal
difficulty_level	21
double_difficulty_level	42

only showing top 1 row

```
In [16]: # Example 2
from pyspark.sql.functions import expr

# Using expressions
df_with_new_count = df_read.withColumn("new_count", expr("count + 1"))

df_with_new_count.show(1,vertical=True)
```

```
-RECORD 0-----
duration                | 0
protocol_type           | tcp
service                 | http
flag                    | SF
src_bytes               | 249
dst_bytes               | 3930
land                   | 0
wrong_fragment          | 0
urgent                  | 0
hot                     | 0
num_failed_logins       | 0
logged_in               | 1
num_compromised         | 0
root_shell              | 0
su_attempted            | 0
num_root                | 0
num_file_creations      | 0
num_shells              | 0
num_access_files        | 0
num_outbound_cmds       | 0
is_host_login           | 0
is_guest_login          | 0
count                   | 1
srv_count               | 1
serror_rate             | 0.0
srv_serror_rate         | 0.0
rerror_rate             | 0.0
srv_rerror_rate         | 0.0
same_srv_rate           | 1.0
diff_srv_rate           | 0.0
srv_diff_host_rate      | 0.0
dst_host_count          | 255
dst_host_srv_count      | 255
dst_host_same_srv_rate  | 1.0
dst_host_diff_srv_rate  | 0.0
dst_host_same_src_port_rate | 0.0
dst_host_srv_diff_host_rate | 0.0
dst_host_serror_rate    | 0.0
dst_host_srv_serror_rate | 0.0
dst_host_rerror_rate    | 0.0
dst_host_srv_rerror_rate | 0.0
classes                 | normal
difficulty_level        | 21
new_count                | 2
only showing top 1 row
```

Aggregating Data

Aggregation operations are essential for summarizing data. PySpark provides various aggregation functions like `sum()`, `avg()`, `min()`, and `max()`.

```
In [17]: from pyspark.sql.functions import avg
```

```
# Create a new column
df_read.select(avg("count")).show()
```

```
+-----+
|      avg(count) |
+-----+
|84.1075547934875|
+-----+
```

Group By Example Using SparkSQL Functions

```
In [18]: df_read.groupby("protocol_type").count().show()
```

```
+-----+-----+
|protocol_type| count|
+-----+-----+
|          tcp|102689|
|          udp| 14993|
|          icmp|  8291|
+-----+-----+
```

Select: Order By

Note that one can use an ordinal number (as in **GROUP BY 1**, meaning group by the first column of the selected table). Also note that the output from GroupBy is not necessarily ordered by any column (the output is in essentially random order)...that's what **ORDER BY** is for.

The ASC means "ascending," which is the default, which is contrasted with DESC for "descending."

```
In [19]: df_read.orderBy("difficulty_level", ascending=False).show(1,vertical=True)
```

```
-RECORD 0-----
duration                | 0
protocol_type           | tcp
service                 | private
flag                    | S0
src_bytes               | 0
dst_bytes               | 0
land                    | 0
wrong_fragment          | 0
urgent                  | 0
hot                     | 0
num_failed_logins       | 0
logged_in               | 0
num_compromised         | 0
root_shell              | 0
su_attempted            | 0
num_root                | 0
num_file_creations      | 0
num_shells              | 0
```

num_access_files	0
num_outbound_cmds	0
is_host_login	0
is_guest_login	0
count	54
srv_count	12
serror_rate	1.0
srv_serror_rate	1.0
rerror_rate	0.0
srv_rerror_rate	0.0
same_srv_rate	0.22
diff_srv_rate	0.11
srv_diff_host_rate	0.0
dst_host_count	255
dst_host_srv_count	16
dst_host_same_srv_rate	0.06
dst_host_diff_srv_rate	0.07
dst_host_same_src_port_rate	0.0
dst_host_srv_diff_host_rate	0.0
dst_host_serror_rate	1.0
dst_host_srv_serror_rate	1.0
dst_host_rerror_rate	0.0
dst_host_srv_rerror_rate	0.0
classes	neptune
difficulty_level	21

only showing top 1 row

Use Schemas as Namespace Organizers

1. Run this command to create a custom schema named "abc": **CREATE SCHEMA ABC;**
2. All tables will have to be created and referenced using abc.yourTableName

Now, solve these questions:

- Find the most frequent 5 difficulty levels in the dataset.
- Find the total number of records where the user was not logged in
- What is the most frequent "attack" in TCP connections? (i.e., records with protocol_type = "tcp").

How to handle AI usage at this point?

- At this point, I'd like to you to continue using your own intellectual abilities in addressing the design questions and solving the problems.
- After you attempt the solution yourself, use an appropriate skill from our agentic skill repositories: <https://github.com/Systems-and-Toolchains-for-AI-Engineers/agentic-skills>
- Compare the output of the skill with your initial solution.
- Does the AI generate additional unnecessary contents in its output?

Readings

- Application of SQL Database, published on Canvas
- Applicationn of PySpark, published on Canvas
- [PySpark Dataframe Guide](#)